Evaluating Dataset Selection Strategies for TCR–Epitope Binding Affinity Prediction

https://github.com/edwin-huang/CSE559TCREpitopeProject

Vignesh Mohan vmohanav@asu.edu
Ali Altaraif aebrahi7@asu.edu
Edwin Huang ehuang19@asu.edu
Skand Parvatikar sparvati@asu.edu

Abstract

There are a vast amount of possible TCRs, and training data for TCR-epitope binding affinity models can be very large. This raises the question of whether we can shrink the training dataset without much loss in performance. We also evaluate whether clustering-based sampling of the dataset results in better performance than randomly sampling the dataset. We use a dataset of 300,016 TCR-epitope pairs with a label of whether it binds and embed it with the catELMo embedder. Then, we train the same downstream neural networks with different fractions of the original training dataset and different subset selection strategies: Random, Cluster-Random (Clustering and sampling uniformly across clusters), and Cluster-Cluster (hierarchical clustering that approximates k-means). We run 5 runs at each strategy and fraction. Performance consistently increases as fraction increases, albeit with diminishing returns, with fraction=0.3 being able to achieve near-full performance. Random sampling consistently matches or slightly outperforms the Cluster-Random and Cluster-Cluster strategies for the tested metrics of AUC, Precision, Recall, and F1 for all fractions. These results indicate that for this task, random sampling is a strong baseline strategy, and simple clustering methods may not improve model generalization without label-aware selection.

1 Introduction

T cell receptor (TCR) recognition of peptide epitopes presented by major histocompatibility complex (MHC) molecules is central to adaptive immune responses. Accurately predicting TCR-epitope binding remains a challenging problem due to the complexities of protein folding and binding interactions, the extreme diversity of TCR variable chains and epitope peptides, and the limited overlap between experimentally observed sequences.

One particular challenge is dataset size. Peptide libraries can range from hundreds of millions to billions of members. Training on a full dataset may be too expensive in some situations, so only a small fraction of the original training data can be used. This raises two questions:

1. How should the most informative data points be selected, and is there a strategy better than random?
2. At what dataset size does model performance begin to plateau?

A baseline approach is to randomly select points from the original dataset. This approach is simple to implement and preserves the original data distribution. However, it is possible that random selection may choose very similar data points, or that a portion of the original dataset happens to be far from any chosen data point.

In this paper, we study if clustering-based subset selection can improve binding predictions for a downstream model compared to random sampling. We also analyze the relationship between performance and fraction. Using catELMo TCR and epitope embeddings, we train the same downstream neural network on various fractions of the training data, chosen by 3 strategies: (1) **Random** sampling, (2) **Cluster-Random**, which samples uniformly across a fixed number of clusters to improve coverage of the dataset, and (3) **Cluster-Cluster**, a hierarchical clustering method that approximates a k-means clustering.

For all three, we find that the performance increases as the fraction increases, with higher fractions yielding slower increases but no definitive plateau, with fraction=0.3 demonstrating good downstream performance. We also find that the clustering-based methods do not significantly outperform random sampling, suggesting that for this task, diversity heuristics may not improve predictive performance by itself.

2 Experimental Setup

2.1 Dataset and Splitting Strategy

We use a dataset of 300,016 TCR-epitope pairs with a balanced 1:1 ratio of positive and negative samples. The dataset is split into 80% training data, and 20% testing data. Given a fraction f, if the training data has size n, we want to develop a strategy to choose $f \cdot n$ data points from the n training data points to maximize the test metrics.

Three strategies were chosen, which we named Random, Cluster-Random, and Cluster-Cluster. In the Random strategy, the $f \cdot n$ new data points are chosen randomly from the training data. In Cluster-Random, the training data is split into 256 clusters, and the training data is taken uniformly from clusters regardless of cluster size. Cluster-Random aims to improve coverage of the dataset. In Cluster-Cluster, the training data is split into 128 clusters, and each cluster is split into k_i further clusters, where the k_i are proportional to cluster sizes and sum to $f \cdot n$. Cluster-Cluster approximates a k-means clustering of $f \cdot n$ centroids from n points, which could not be done due to runtime constraints. All clustering is performed on catELMo embeddings.

2.2 Split Evaluation Strategy

To evaluate our data splits, we embedded input TCR and Epitope peptides with the catELMo embedder[1]. Then, for each split strategy and fraction, we train a downstream model: The TCR and epitope embeddings are each projected onto a 128-dimension hidden layer, then concatenated and projected to a 2048 dimension layer, and projected to the output layer. In each projection, we used a Linear layer, then Batch normalization, then SiLU as the activation function, then Dropout with rate 0.3. This is the same architecture as the downstream model used by catELMo [1], except with Dropout moved after the activation function, which improved performance, and with biases in the linear layers removed, as Batch Normalization removes these biases right after. The downstream model has 793,089 parameters.

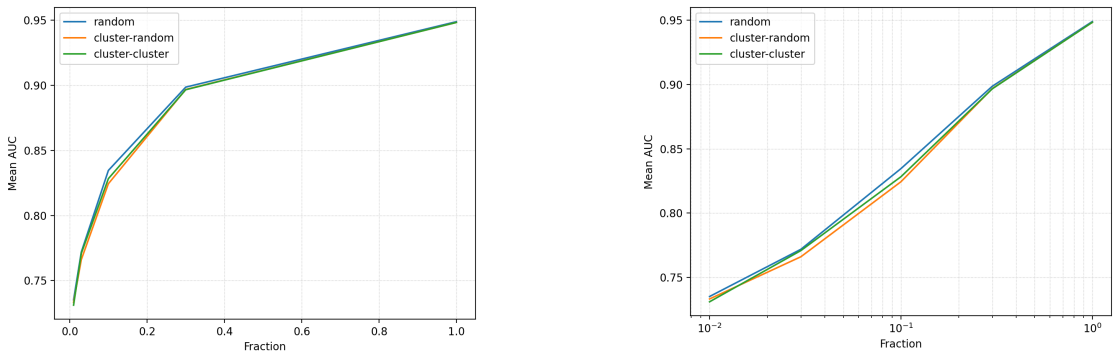
Metrics for AUC, Precision, Recall, and F1 score are averaged over five repeated runs for each strategy and fraction, with their mean and standard deviation reported. Fraction=1.0 is included for completeness.

3 Results

Table 1 summarizes the AUC, Precision, Recall, and F1 metrics for all evaluated models. The metrics in the Random split are consistently higher than those in the other splits.

Table 1: Performance comparison across models (mean ± std over 5 runs).

Model (AUC %)	0.01	0.03	0.1	0.3	1
Random	**73.52 ± 0.52**	**77.19 ± 0.41**	**83.46 ± 0.65**	**89.86 ± 0.24**	**94.89 ± 0.14**
Cluster-Random	73.33 ± 0.25	76.61 ± 0.31	82.44 ± 0.52	89.68 ± 0.06	94.83 ± 0.11
Cluster-Cluster	73.11 ± 0.28	77.09 ± 0.35	82.83 ± 0.85	89.66 ± 0.13	94.83 ± 0.15
Model (Precision %)	**0.01**	**0.03**	**0.1**	**0.3**	**1**
Random	**68.45 ± 2.00**	69.77 ± 0.37	**75.15 ± 0.56**	**80.77 ± 0.33**	**87.09 ± 0.18**
Cluster-Random	68.15 ± 0.54	**70.15 ± 0.41**	74.23 ± 0.66	80.56 ± 0.11	86.97 ± 0.20
Cluster-Cluster	66.29 ± 0.88	68.76 ± 0.56	73.95 ± 0.84	80.53 ± 0.20	86.98 ± 0.18
Model (Recall %)	**0.01**	**0.03**	**0.1**	**0.3**	**1**
Random	66.45 ± 0.45	**69.56 ± 0.39**	**74.92 ± 0.57**	**80.72 ± 0.29**	**87.08 ± 0.18**
Cluster-Random	**66.86 ± 0.23**	69.27 ± 0.21	74.07 ± 0.58	80.55 ± 0.10	86.96 ± 0.20
Cluster-Cluster	65.89 ± 0.73	68.68 ± 0.56	73.94 ± 0.84	80.45 ± 0.17	86.97 ± 0.18
Model (F1 %)	**0.01**	**0.03**	**0.1**	**0.3**	**1**
Random	65.58 ± 1.07	**69.48 ± 0.40**	**74.87 ± 0.58**	**80.71 ± 0.29**	**87.08 ± 0.18**
Cluster-Random	**66.26 ± 0.45**	68.94 ± 0.26	74.03 ± 0.56	80.55 ± 0.10	86.96 ± 0.20
Cluster-Cluster	65.68 ± 0.75	68.65 ± 0.56	73.94 ± 0.84	80.44 ± 0.17	86.97 ± 0.18



(a) AUC vs fraction

(b) AUC vs fraction (log scale)

Figure 1: Model performance vs training data fraction for various strategies.

4 Hyperparameter Choice

We did a search for various hyperparameters, using the catELMo parameters as a baseline[1]. We chose hyperparameters based on performance at low and high fractions. Values in bold were ultimately chosen. Dropout: {0.0, 0.1, 0.2, **0.3**, 0.4, 0.5}, 1st hidden layer size: {**128**, 256, 512, 1024, 2048}, 2nd hidden layer size: {256, 512, 1024, **2048**}, number of hidden layers after concatenation: {**1**, 2, 3}, learning rate: {1e-4, **3e-4**, 1e-3, 3e-3}, batch size: {32, 128, 512, **1024**}, optimizer choice: {**Adam**, AdamW}, optimizer beta2: {0.95, **0.999**}, Cluster-Random initial split: {64, 128, **256**}, Cluster-Cluster initial split: {64, **128**, 256}. Of note is the batch size at 1024, which did not decrease AUC performance while massively speeding up model training.

5 Discussion

AUC consistently increases with increasing fraction, with no notable plateau. Using fraction=0.3 results in a slightly higher AUC to log(data) ratio. Interestingly, training loss increases at first from fraction=0.01 to 0.03, but then decreases, which may indicate early overfitting.

Random sampling of the original dataset consistently yields similar or slightly higher performance compared to clustering-based strategies across all data fractions. This may suggest that the inherent density and distribution of the TCR-epitope sequence space are important for the model to learn the "recognition code" effectively. Another explanation is that the original dataset is already large, diverse, and well distributed, so our clustering-based approaches may not have any positive impact, which is plausible given that the dataset is in embedding space.

The Cluster-Random strategy aims to improve dataset coverage by spreading data points uniformly across clusters. However, since the test distribution is the same as the original training distribution, this strategy makes the new distribution significantly different than the test distribution. By assigning an equal number of points to small and large clusters, the model cannot learn as much in the larger clusters. Consequently, performance may drop because these high-density regions are the most prevalent in the test set.

In the Cluster-Cluster approach, we attempted to approximate a k-means clustering of centroids. This strategy tries to make sure no data points are too close to each other. However, this is not guaranteed to improve the final AUC scores. Looking at the ATM-TCR paper where the dataset is sourced from[2], negative examples are created by taking a positive data point and recombining its TCR with another TCR. This means that data points that are very close to each other may have different labels, which may help the model distinguish between positive and negative classes.

6 Conclusion

This study evaluates various methods for reducing dataset size when training a TCR/Epitope binding classifier. Our results indicate that while increasing dataset size does increase performance, none of the proposed clustering strategies provided a clear improvement over choosing random points from the dataset. We hypothesize that random sampling is difficult to outperform because it preserves the natural data distribution, ensuring the model is trained on the same statistical patterns it will encounter during evaluation.

References

[1] Pengfei Zhang, Seojin Bang, Michael Cai, and Heewook Lee. "Context-Aware Amino Acid Embedding Advances Analysis of TCR-Epitope Interactions". In: (Apr. 2024). DOI: 10.7554/elife.88837.2. URL: http://dx.doi.org/10.7554/eLife.88837.2.

[2] Michael Cai, Seojin Bang, Pengfei Zhang, and Heewook Lee. "ATM-TCR: TCR-Epitope Binding Affinity Prediction Using a Multi-Head Self-Attention Model". In: *Frontiers in Immunology* Volume 13 - 2022 (2022). ISSN: 1664-3224. DOI: 10.3389/fimmu.2022.893247. URL: https://www.frontiersin.org/journals/immunology/articles/10.3389/fimmu.2022.893247.